

Day 4 Evidence Workbooklet — Instructor Key

Comparisons and Boundary Cases

Engineering practice → threshold rule → Python comparison → boundary evidence → support next step

Instructor key. Same layout as student copy; solutions filled in response boxes. Print format: duplex, left-stapled packet.

Name		Section		Date	
Score	____/44		Mastery check	secure / developing / needs support	

Yesterday the pump-flow intake values were converted from text into numeric fields. Today the team uses those typed values to check a quality boundary: a flow reading is acceptable if it is inside a target band. The exact boundary rows matter because a reading exactly at the lower or upper limit may be allowed depending on the operator.

The problem today is comparison evidence: below, at, or above a limit; inclusive versus exclusive rules; and an exact sentence that explains a boundary case.

Engineering task	Check whether converted pump-flow readings meet an acceptable range before the team uses them in a go/no-go gate.
Computational move	Evaluate one comparison at a time and explain exact-boundary cases without jumping ahead to compound decisions.
Python focus	<, <=, >, >=, ==, !=, comparison result, inclusive limit, exclusive limit.
Evidence to keep	reading value, boundary value, operator, true/false result, and a sentence explaining whether the boundary is included.

Day 4 boundary rules

1. A comparison expression produces **True** or **False**.
2. <= and >= include equality at the boundary.
3. < and > do not include equality at the boundary.
4. == checks equality; it does not assign a value.
5. A reliable threshold note names the value, the limit, the operator, and the result.

1 Read the threshold notebook one comparison at a time

[recognize](#)[predict](#)

Use the converted numeric fields from the intake record. Do not combine the comparisons yet; Day 5 will do that.

```
1 # Converted numeric fields from the intake record
2 target_flow_lpm = 20.0
3 low_limit_lpm = 19.6
4 high_limit_lpm = 20.4
5 reading_low_edge = 19.6
6 reading_middle = 20.0
7 reading_high_edge = 20.4
8 reading_high_miss = 20.5
9
10 # Individual comparison checks
11 reading_low_edge >= low_limit_lpm
12 reading_low_edge > low_limit_lpm
13 reading_high_edge <= high_limit_lpm
14 reading_high_edge < high_limit_lpm
15 reading_high_miss <= high_limit_lpm
16 reading_middle == target_flow_lpm
```

Pts.	Prompt	My evidence
1	Which variables define the lower and upper allowed flow limits?	Key: low_limit_lpm and high_limit_lpm.
1	Which reading is exactly equal to the lower limit?	Key: reading_low_edge = 19.6.
1	Which reading is exactly equal to the upper limit?	Key: reading_high_edge = 20.4.
1	Which reading is above the upper limit?	Key: reading_high_miss = 20.5.

2 Predict comparison results at exact boundaries

predict explain

Complete the table. Explain the result using the operator, not just the number.

Pts.	Comparison expression	True or False?	Boundary reason
2	<code>reading_low_edge >= low_limit_lpm</code>	Key: True	Key: Equal to lower limit; >= includes equality.
2	<code>reading_low_edge > low_limit_lpm</code>	Key: False	Key: Equal is not greater than.
2	<code>reading_high_edge <= high_limit_lpm</code>	Key: True	Key: Equal to upper limit; <= includes equality.
2	<code>reading_high_edge < high_limit_lpm</code>	Key: False	Key: Equal is not less than.

Common trap to check

A reading exactly at the limit is included by `<=` or `>=`, but it is not included by `<` or `>`. The operator decides the boundary, not the word "limit" by itself.

3 Separate equality from assignment and mismatch checks

recognize

predict

Python uses == for equality checks and = for assignment. This page keeps comparison evidence separate from stored state.

Pts.	Expression or line	Result or effect	What evidence does it create?
2	reading_middle == target_flow_lpm	Key: True	Key: Equality comparison; no assignment.
2	reading_high_miss != target_flow_lpm	Key: True	Key: 20.5 is not equal to 20.0.
2	reading_middle = target_flow_lpm	Key: assignment	Key: Stores 20.0 in reading_middle; not a comparison.
2	reading_high_miss <= high_limit_lpm	Key: False	Key: 20.5 is above 20.4.

Pts.	Prompt	My response
2	Why is == a safer symbol than the word "is" when writing comparison evidence?	Key: == names an equality comparison explicitly; "is" can be ambiguous in prose.
2	Why should a threshold note not say only "near the target"?	Key: It must name the value, limit, operator, and whether equality is included.

4 Build a boundary table before combining conditions

[trace](#) [explain](#)

Complete the boundary table. The last column should be a short engineering sentence, not only **True** or **False**.

Pts.	Reading	Lower check: ≥ 19.6	Upper check: ≤ 20.4	Boundary evidence sentence
2	19.5	Key: False	Key: True	Key: Below lower limit; not allowed.
2	19.6	Key: True	Key: True	Key: At lower limit; inclusive lower boundary allows it.
2	20.4	Key: True	Key: True	Key: At upper limit; inclusive upper boundary allows it.
2	20.5	Key: True	Key: False	Key: Above upper limit; not allowed.

Bridge to Day 5

Tomorrow the team will combine separate comparison results into a go/no-go expression. Today the important habit is to make each comparison traceable first: value, operator, limit, result, and boundary sentence.

5 Debug a boundary claim

debug test explain

A teammate writes this note after checking the lower edge reading:

Boundary note to debug

The 19.6 L/min reading fails because it is not greater than the lower limit of 19.6 L/min.

Pts.	Prompt	My response
2	What part of the teammate's sentence uses a real comparison result?	Key: <code>19.6 > 19.6</code> is False.
2	What rule did the teammate use that may not match the allowed boundary?	Key: They used an exclusive <code>></code> rule. If the allowed boundary is inclusive, use <code>>=</code> .
2	Write the inclusive comparison that would allow the exact lower-limit reading.	Key: <code>reading_low_edge >= low_limit_lpm</code> .
2	Rewrite the boundary note so it names the operator and explains whether the exact limit is included.	Key: Using <code>>=</code> , <code>19.6 >= 19.6</code> is True, so the exact lower-limit reading is allowed.

Boundary-test habit

When a threshold rule matters, test at least three rows: just below the boundary, exactly at the boundary, and just above the boundary. Exact-boundary rows catch many comparison mistakes.

6 Mastery check and support next step

[transfer](#)[explain](#)

Use your own evidence from this workbooklet. Do not write a generic answer.

Pts.	Prompt	My response
2	Give one example where changing from <code>></code> to <code>>=</code> changes the result at an exact boundary.	Key: <code>reading_low_edge > low_limit_lpm</code> is False, but <code>reading_low_edge >= low_limit_lpm</code> is True.
2	In one or two sentences, explain why Day 3 type/cast evidence is needed before Day 4 boundary checks.	Key: Boundary comparisons require numeric values. Day 3 casting makes sure text fields have become floats before comparison.

My mastery check

Mark the strongest statement that is true after this workbooklet.

☐	Secure: I can evaluate comparison operators and explain exact lower or upper boundary cases.
☐	Developing: I can evaluate some comparisons, but I still need support with inclusive versus exclusive limits.
☐	Needs support: I need a smaller example before I can trust my boundary-check evidence.

My next support move

My issue is mostly: setup / Python syntax / tracing / operator choice / boundary cases / confidence / time.

The first boundary where I got uncertain was: _____

My next small action is: ask a partner / ask instructor / rebuild the boundary table / test just-below-at-above rows / try the recovery version.

Workbooklet target: I can evaluate Python comparison operators, explain exact-boundary cases, and prepare separate comparison evidence before Boolean combinations.

Copyright © 2026 Lance L.A. White. All rights reserved.

Bridge Day 4 VIP Evidence Handoff

INSTRUCTOR KEY • Contract 07a04826c975 • Accessible v5 successor

Why engineers use this page

Carry below-at-above boundary evidence into complete comparison and ordered-branch programs without requiring the supplied-loop activity yet.

Decision boundary: Code is useful only when its stored state, visible result, assumptions, units, limits, and tests support a defensible engineering interpretation.

Construct readiness before independent work

New authoring tools: comparison expression, boolean operator, if elif else, abs call, counter update

Carried authoring tools: assignment, print call, numeric literal, arithmetic expression, input call, int conversion, float conversion, f string

Supplied-code exposure only: for loop, list traversal

An exposure-only construct may be traced in complete supplied code, but the independent task will not require students to invent it before its authoring day.

Notebook cells and task constructs

Activity	Work	Stable cells	Required authoring constructs
18.13	Compare With Tolerance	18-13-work / 18-13-transfer / 18-13-cold	comparison expression, f string, float conversion, input call
18.15	Lamination Fee	18-15-work / 18-15-transfer / 18-15-cold	comparison expression, f string, if elif else, input call, int conversion
18.16	Temperature Status	18-16-work / 18-16-transfer / 18-16-cold	comparison expression, f string, float conversion, if elif else, input call
18.17	Pickup Window	18-17-work / 18-17-transfer / 18-17-cold	boolean operator, comparison expression, f string, if elif else, input call, int conversion

Readiness evidence

1. Choose one mapped activity and write the below, exact-boundary, and above test values before coding.

Model response: Credit a valid three-case boundary set tied to the activity, including tolerance equality, page thresholds, maximum plus five, or the travel-time boundary.

2. For the selected activity, list input conversion, named comparison evidence, first matching branch, and exact formatted output.

Model response: The chain must begin with numeric conversion, preserve the comparison result, respect branch priority, and end with the specified label and formatting.

Timing and help trigger

Record a first prediction before running. If you still lack an executable plan after about 8 focused minutes, use the linked ThinkCSPy refresher or the supplied model. If two attempts fail for the same named requirement, write the exact mismatch and ask a peer mentor or instructor a targeted question. Timing is diagnostic evidence, not a speed grade. **Start or elapsed minutes:** _____ **My help trigger:** _____

Engineering Evidence Record

Complete one record from a model, transfer, or Independent Program Studio build. Generic statements are not sufficient; use actual values, a real revision, and one concrete next test.

Evidence field	Record
Prediction	A specific expected state or output before execution.
Observed Result	The actual state or output, including a value or exact text.
Revision Reason	The defect or mismatch and the change that addresses it.
Engineering Meaning	How the evidence changes or supports the engineering decision.
Units Or Not Applicable	The physical unit, or the evidence type when no unit applies.
Assumption	One condition accepted as true for this model or rule.
Model Limit	One situation the result does not justify or predict.
Next Test	A concrete boundary, invalid, or alternate case with an expected result.
Help Trigger	The exact unresolved point that would trigger a targeted question.
Minutes Used	Approximate minutes from first prediction through revision.

Notebook and submission procedure

1. Attempt, predict, run, inspect, and revise before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those visibly labeled Studio cells are scored for independent mastery.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.

Generated from the canonical VIP construct and evidence contract; workbook planning and executable notebook work remain distinct.